



Data Handling with Python (pandas)

Part 1

Chapter 1

Introduction to Pandas

Welcome to the world of data handling with Python. In today's world, data is everywhere — from student records in a university to sales reports in a company, from patient information in a hospital to customer transactions in an online store. Managing, analyzing, and making sense of this data is one of the most important skills in the modern workplace.

Python, as we learned earlier, is one of the most popular programming languages. But Python alone does not come with built-in tools for advanced data handling. This is where **pandas** comes in — a powerful, open-source library specifically designed for working with structured data in Python.

1.1 What is Pandas?

pandas

An open-source Python library that provides fast, flexible, and expressive data structures designed to make working with structured (tabular) data easy and intuitive. The name “pandas” is derived from “Panel Data” — a term used in statistics and econometrics for multi-dimensional structured data sets.

In simple terms, pandas allows you to work with data in a table format — similar to how you work with data in Microsoft Excel or Google Sheets — but with the full power of Python programming behind it. You can load data from files, clean it, filter it, sort it, perform calculations, and export the results — all using just a few lines of code.

Think of pandas as Python’s answer to Excel. While Excel is a visual tool where you click and drag, pandas lets you do the same operations programmatically — which means you can process thousands or even millions of rows in seconds, automate repetitive tasks, and build reusable analysis pipelines.

Key Capabilities of Pandas

Capability	What It Means
Data Loading	Read data from CSV, Excel, JSON, SQL databases, and more
Data Cleaning	Handle missing values, remove duplicates, fix data types
Data Selection	Filter rows and columns based on conditions
Data Transformation	Sort, group, merge, reshape, and pivot data
Data Analysis	Calculate statistics like mean, sum, count, and more
Data Export	Save processed data back to CSV, Excel, JSON, or other formats

Why Pandas is Important

- Before pandas existed, working with tabular data in Python required writing complex code with basic lists and dictionaries. Pandas simplified this dramatically by introducing two powerful data structures:
- Series** — A one-dimensional labeled array (like a single column in a spreadsheet).
 - DataFrame** — A two-dimensional labeled data structure (like an entire spreadsheet or SQL table).

These two structures form the foundation of everything you will do with pandas. We will explore each of them in detail in this chapter.

Where is Pandas Used?

Industry	Use Case
Finance & Banking	Analyzing stock prices, portfolio management, risk assessment
Healthcare	Managing patient records, analyzing clinical trial data
E-Commerce	Customer behavior analysis, sales trend forecasting
Education	Student performance tracking, attendance analysis
Government	Census data processing, public policy research
Marketing	Campaign performance analysis, customer segmentation



Note

pandas is not a separate programming language. It is a Python library — meaning it is a collection of pre-written Python code that you import into your own programs. You still write Python code; pandas simply gives you additional tools to work with data more efficiently.

ETIHANS
CLOUD & AI ACADEMY



Chapter 2

Installing and Importing Pandas

Before you can use pandas in your Python programs, you need to install it on your computer and then import it into your script. This section walks you through both steps.

2.1 Installing Pandas

pandas is not included in Python by default. It is an external library that must be installed separately. The most common way to install pandas is using **pip** — Python’s built-in package manager.

pip

The standard package installer for Python. It allows you to download and install third-party libraries (like pandas) from the Python Package Index (PyPI) — a repository of thousands of free Python packages.

Method 1: Installing via Command Prompt / Terminal

Open the Command Prompt (Windows) or Terminal (macOS/Linux) and type the following command:

```
pip install pandas
```

Figure 2.1 — Installing pandas using pip

If you already have pandas installed and want to upgrade to the latest version, use:

```
pip install --upgrade pandas
```

Figure 2.2 — Upgrading pandas to the latest version

Method 2: Installing via Anaconda

If you are using the **Anaconda distribution** (a popular Python distribution for data science), pandas comes pre-installed. You can verify or install it using:

```
conda install pandas
```

Figure 2.3 — Installing pandas using conda

Verifying the Installation

After installation, verify that pandas is installed correctly by opening a Python environment and running:

```
import pandas
print(pandas.__version__)
```

Figure 2.4 — Verifying the pandas installation

Output
2.2.3



Note

The version number displayed on your screen may differ from the one shown above. As long as no error appears when you run 'import pandas', the installation is successful.

2.2 Importing Pandas

Once pandas is installed, you need to **import** it into your Python script before you can use it. Importing tells Python: “I want to use the pandas library in this program.”

Standard Import

The most common and recommended way to import pandas is with an alias:

```
import pandas as pd
```

Figure 2.5 — Standard pandas import with alias

Here, **pd** is the alias. This is the universally accepted convention — almost every tutorial, textbook, and professional codebase uses **pd** as the alias for pandas. From this point onward, instead of writing **pandas.something()**, you will write **pd.something()**.

Alternative Import Methods

```
# Method 1: Import without alias (not recommended for daily use)
import pandas

# Method 2: Import specific components only
from pandas import DataFrame, Series
```

Figure 2.6 — Alternative import methods

**Pro Tip**

Always use 'import pandas as pd' in your programs. This is the industry standard, and using any other convention will make your code harder for others to read and understand.

What Happens if Pandas is Not Installed?

If you try to import pandas without installing it first, Python will display the following error:

```
import pandas as pd
```

Output

```
ModuleNotFoundError: No module named 'pandas'
```

If you see this error, go back to section 2.1 and follow the installation steps.

Common Libraries Installed Alongside Pandas

Library	Purpose
NumPy	Provides support for numerical operations and arrays. Pandas is built on top of NumPy.
python-dateutil	Helps pandas work with dates and times effectively.
pytz	Provides timezone support for date/time operations.

**Common Error**

Do not name your Python file 'pandas.py'. If you do, Python will try to import your file instead of the actual pandas library, causing an error. Always choose a different filename such as 'my_analysis.py' or 'data_project.py'.

Chapter 3

Pandas Series

Now that you have pandas installed and imported, it is time to learn about the first of the two core data structures in pandas — the **Series**.

3.1 What is a Pandas Series?

Series	A one-dimensional labeled array in pandas that can hold data of any type — integers, floats, strings, or booleans. Each value in a Series has a corresponding label called an index.
---------------	--

You can think of a Series as a single column from a spreadsheet. It has a list of values, and each value has a label (index) that identifies its position.

For example, consider a list of marks scored by students in an exam:

Index	Value (Marks)
0	85
1	92
2	78
3	90
4	88

This is essentially what a pandas Series looks like — a column of values with an index on the left. By default, pandas assigns numerical indices starting from 0.

3.2 Creating a Pandas Series

There are several ways to create a Series. Let us explore each one with examples.

Creating a Series from a Python List

The most common way to create a Series is by passing a Python list to the **pd.Series()** function.

```
import pandas as pd

# Creating a Series from a list of marks
marks = [85, 92, 78, 90, 88]
series_marks = pd.Series(marks)

print(series_marks)
```

Figure 3.1 — Creating a Series from a Python list

Output

0	85
1	92
2	78
3	90
4	88

dtype: int64

Notice that pandas automatically assigned indices 0, 1, 2, 3, 4 to the five values. The last line **dtype: int64** tells you the data type of the values stored in the Series — in this case, 64-bit integers.

Creating a Series with Custom Index

Instead of the default numerical index, you can assign meaningful labels using the **index** parameter.

```
import pandas as pd

# Creating a Series with student names as index
marks = [85, 92, 78, 90, 88]
students = ["Alice", "Bob", "Charlie", "David", "Eve"]

series_marks = pd.Series(marks, index=students)

print(series_marks)
```

Figure 3.2 — Series with custom index labels

Output

Alice	85
Bob	92


```
Charlie    78
David      90
Eve        88
dtype: int64
```

Creating a Series from a Dictionary

You can also create a Series from a Python dictionary. The dictionary keys automatically become the index labels.

```
import pandas as pd

# Creating a Series from a dictionary
employee_salary = {
    "Alice"    : 55000,
    "Bob"      : 62000,
    "Charlie"  : 48000,
    "David"    : 71000
}

series_salary = pd.Series(employee_salary)

print(series_salary)
```

Figure 3.3 — Creating a Series from a dictionary

```
Output
Alice    55000
Bob      62000
Charlie  48000
David    71000
dtype: int64
```

Creating a Series with a Scalar Value

If you pass a single value with an index, pandas repeats that value for every index position.

```
import pandas as pd

# Creating a Series with a constant value
constant_series = pd.Series(0, index=["Math", "Science", "English", "History"])
```

```
print(constant_series)
```

Figure 3.4 — Creating a Series with a scalar (constant) value

Output

```
Math      0
Science   0
English    0
History    0
dtype: int64
```



Pro Tip

Creating a Series with a constant value is useful when you want to initialize a data structure with default values — for example, setting all student attendance records to 0 at the start of a semester.

Naming a Series

You can assign a name to a Series using the **name** parameter. This is especially useful when the Series will later become part of a DataFrame.

```
import pandas as pd

marks = pd.Series([85, 92, 78], index=["Alice", "Bob", "Charlie"], name="Exam Marks")

print(marks)
```

Figure 3.5 — Creating a named Series

Output

```
Alice      85
Bob         92
Charlie     78
Name: Exam Marks, dtype: int64
```

Chapter 4

Handling Pandas Series

Now that you know how to create a Series, let us learn how to work with it — accessing values, performing operations, and using built-in methods to analyze data.

4.1 Accessing Elements in a Series

Accessing by Index Position

```
import pandas as pd

marks = pd.Series([85, 92, 78, 90, 88], index=["Alice", "Bob", "Charlie",
"David", "Eve"])

# Access by position
print(marks[0])      # First element
print(marks[2])      # Third element
```

Figure 4.1 — Accessing Series elements by position

```
Output
85
78
```

Accessing by Label

```
# Access by label
print(marks["Alice"])    # 85
print(marks["David"])    # 90
```

Figure 4.2 — Accessing Series elements by label

```
Output
85
90
```

Accessing Multiple Elements (Slicing)

```
# Slicing by position (end index excluded)
print(marks[1:4])

# Slicing by label (both endpoints included)
print(marks["Bob":"David"])
```

Figure 4.3 — Slicing a Series

Output

```
Bob      92
Charlie  78
David    90
dtype: int64

Bob      92
Charlie  78
David    90
dtype: int64
```

**Note**

When slicing by position (`marks[1:4]`), the end index is excluded. When slicing by label (`marks["Bob":"David"]`), both the start and end labels are included. This is an important difference to remember.

4.2 Series Attributes

A Series has several useful attributes that give you information about the data it contains.

```
import pandas as pd

marks = pd.Series([85, 92, 78, 90, 88], index=["Alice", "Bob", "Charlie",
"David", "Eve"])

print("Values :", marks.values)
print("Index  :", marks.index)
print("Data Type:", marks.dtype)
print("Size   :", marks.size)
print("Shape  :", marks.shape)
print("Is Empty:", marks.empty)
```

Figure 4.4 — Series attributes

Output

```
Values   : [85 92 78 90 88]
Index    : Index(['Alice', 'Bob', 'Charlie', 'David', 'Eve'], dtype='object')
Data Type: int64
Size     : 5
Shape    : (5,)
Is Empty: False
```

4.3 Arithmetic Operations on a Series

You can perform mathematical operations on an entire Series at once. The operation is applied to every element — this is called **vectorized operations**.

```
import pandas as pd

prices = pd.Series([500, 800, 1200, 350], index=["Shirt", "Jacket", "Shoes",
"Cap"])

# Increase all prices by 10%
new_prices = prices * 1.10
print(new_prices)

# Subtract a flat discount of 50
discounted = prices - 50
print(discounted)
```

Figure 4.5 — Arithmetic operations on a Series

Output

```
Shirt      550.0
Jacket     880.0
Shoes     1320.0
Cap        385.0
dtype: float64

Shirt      450
Jacket     750
Shoes     1150
Cap        300
dtype: int64
```

4.4 Statistical Methods on a Series

Pandas provides built-in methods to quickly compute common statistics on your data.

```
import pandas as pd

marks = pd.Series([85, 92, 78, 90, 88], index=["Alice", "Bob", "Charlie",
"David", "Eve"])

print("Sum      :", marks.sum())
print("Mean     :", marks.mean())
print("Median   :", marks.median())
print("Min      :", marks.min())
print("Max      :", marks.max())
print("Std Dev  :", marks.std())
print("Count    :", marks.count())
```

Figure 4.6 — Statistical methods on a Series

Output

```
Sum      : 433
Mean     : 86.6
Median   : 88.0
Min      : 78
Max      : 92
Std Dev  : 5.3665...
Count    : 5
```

The describe() Method

The **describe()** method provides a quick summary of all major statistics in one call.

```
print(marks.describe())
```

Figure 4.7 — Using describe() for a statistical summary

Output

```
count      5.000000
mean       86.600000
std         5.366563
min        78.000000
25%        85.000000
50%        88.000000
75%        90.000000
```

```
max      92.000000
dtype: float64
```

4.5 Filtering Data in a Series

You can filter data in a Series using conditions. This is called **Boolean indexing** — you write a condition, and pandas returns only the elements that satisfy it.

```
import pandas as pd

marks = pd.Series([85, 92, 78, 90, 88], index=["Alice", "Bob", "Charlie",
"David", "Eve"])

# Students who scored above 85
print(marks[marks > 85])

# Students who scored exactly 90
print(marks[marks == 90])
```

Figure 4.8 — Filtering a Series using conditions

```
Output
Bob      92
David    90
Eve      88
dtype: int64

David    90
dtype: int64
```

4.6 Modifying a Series

```
import pandas as pd

marks = pd.Series([85, 92, 78, 90, 88], index=["Alice", "Bob", "Charlie",
"David", "Eve"])

# Change Charlie's marks
marks["Charlie"] = 82
```

```
# Add a new student
marks["Frank"] = 95

# Delete a student
marks = marks.drop("Eve")

print(marks)
```

Figure 4.9 — Modifying elements in a Series

Output

Alice	85
Bob	92
Charlie	82
David	90
Frank	95

dtype: int64

4.7 Handling Missing Data in a Series

In real-world datasets, some values are often missing. Pandas represents missing data as **NaN** (Not a Number). You can detect and handle missing values using built-in methods.

```
import pandas as pd
import numpy as np

# Series with missing values
data = pd.Series([10, np.nan, 30, np.nan, 50], index=["A", "B", "C", "D", "E"])

# Detect missing values
print(data.isnull())

# Count missing values
print("Missing values:", data.isnull().sum())

# Fill missing values with 0
filled = data.fillna(0)
print(filled)

# Drop rows with missing values
cleaned = data.dropna()
```



```
print(cleaned)
```

Figure 4.10 — Handling missing data in a Series

Output

```
A    False
B     True
C    False
D     True
E    False
dtype: bool
```

Missing values: 2

```
A    10.0
B     0.0
C    30.0
D     0.0
E    50.0
dtype: float64
```

```
A    10.0
C    30.0
E    50.0
dtype: float64
```

Pro Tip

When working with real data, always check for missing values before performing calculations. The `isnull().sum()` method is a quick way to find how many values are missing.

Chapter 5

Pandas DataFrame

While a Series represents a single column of data, most real-world datasets have multiple columns. For example, a student record has a name, age, course, and marks — all organized in rows and columns. This is where the **DataFrame** comes in.

5.1 What is a Pandas DataFrame?

DataFrame	A two-dimensional, size-mutable, and labeled data structure in pandas — like a table or spreadsheet. It consists of rows and columns, where each column can hold data of a different type. A DataFrame is essentially a collection of Series that share the same index.
------------------	---

A DataFrame looks like this:

	Name	Age	Course	GPA
0	Alice	21	B.Tech CS	8.5
1	Bob	22	B.Tech IT	7.9
2	Charlie	20	BCA	9.1
3	David	23	B.Tech CS	8.0

The numbers on the left (0, 1, 2, 3) are the **row index**. The labels at the top (Name, Age, Course, GPA) are the **column names**. Each column is a pandas Series.

5.2 Creating a DataFrame

Creating a DataFrame from a Dictionary

The most common way to create a DataFrame is from a Python dictionary, where each key becomes a column name and each value (a list) becomes the column data.

```
import pandas as pd

# Creating a DataFrame from a dictionary
```

```
data = {
    "Name" : ["Alice", "Bob", "Charlie", "David"],
    "Age" : [21, 22, 20, 23],
    "Course" : ["B.Tech CS", "B.Tech IT", "BCA", "B.Tech CS"],
    "GPA" : [8.5, 7.9, 9.1, 8.0]
}

df = pd.DataFrame(data)

print(df)
```

Figure 5.1 — Creating a DataFrame from a dictionary

Output

	Name	Age	Course	GPA
0	Alice	21	B.Tech CS	8.5
1	Bob	22	B.Tech IT	7.9
2	Charlie	20	BCA	9.1
3	David	23	B.Tech CS	8.0

Creating a DataFrame from a List of Lists

You can also create a DataFrame from a list of lists, where each inner list represents a row. You must specify the column names separately.

```
import pandas as pd

data = [
    ["Alice", 21, "B.Tech CS", 8.5],
    ["Bob", 22, "B.Tech IT", 7.9],
    ["Charlie", 20, "BCA", 9.1],
    ["David", 23, "B.Tech CS", 8.0]
]

columns = ["Name", "Age", "Course", "GPA"]

df = pd.DataFrame(data, columns=columns)

print(df)
```

Figure 5.2 — Creating a DataFrame from a list of lists

Output

	Name	Age	Course	GPA
0	Alice	21	B.Tech CS	8.5
1	Bob	22	B.Tech IT	7.9
2	Charlie	20	BCA	9.1
3	David	23	B.Tech CS	8.0

Creating a DataFrame from a List of Dictionaries

Each dictionary in the list represents one row, with keys as column names.

```
import pandas as pd

data = [
    {"Name": "Alice", "Age": 21, "GPA": 8.5},
    {"Name": "Bob", "Age": 22, "GPA": 7.9},
    {"Name": "Charlie", "Age": 20, "GPA": 9.1}
]

df = pd.DataFrame(data)

print(df)
```

Figure 5.3 — Creating a DataFrame from a list of dictionaries

Output

	Name	Age	GPA
0	Alice	21	8.5
1	Bob	22	7.9
2	Charlie	20	9.1

Creating a DataFrame with Custom Index

```
import pandas as pd

data = {
    "Name" : ["Alice", "Bob", "Charlie"],
    "Marks" : [85, 92, 78]
}

df = pd.DataFrame(data, index=["S001", "S002", "S003"])
```

```
print(df)
```

Figure 5.4 — DataFrame with custom index

Output

	Name	Marks
S001	Alice	85
S002	Bob	92
S003	Charlie	78

**Note**

The custom index can be any list of unique labels — student IDs, roll numbers, employee codes, etc. This makes your data more meaningful and easier to look up.

ETIHANS
CLOUD & AI ACADEMY



Chapter 6

Handling Pandas DataFrame

Now that you can create DataFrames, let us learn how to inspect, modify, and manipulate them.

6.1 Inspecting a DataFrame

When you load or create a DataFrame, the first thing you should do is inspect it — understand its size, structure, and the kind of data it contains.

```
import pandas as pd

data = {
    "Name"    : ["Alice", "Bob", "Charlie", "David", "Eve"],
    "Age"     : [21, 22, 20, 23, 21],
    "Course"  : ["B.Tech CS", "B.Tech IT", "BCA", "B.Tech CS", "BCA"],
    "GPA"     : [8.5, 7.9, 9.1, 8.0, 8.8]
}

df = pd.DataFrame(data)

# View first few rows
print(df.head())

# View shape (rows, columns)
print("Shape:", df.shape)

# View column names
print("Columns:", df.columns.tolist())

# View data types
print(df.dtypes)
```

Figure 6.1 — Inspecting a DataFrame

Method / Attribute	What It Returns
df.head(n)	First n rows (default 5)
df.tail(n)	Last n rows (default 5)

df.shape	Tuple of (rows, columns)
df.columns	List of column names
df.dtypes	Data type of each column
df.info()	Summary: columns, data types, non-null counts, memory usage
df.describe()	Statistical summary for numerical columns
df.size	Total number of elements (rows × columns)
df.empty	True if the DataFrame has no data

6.2 Adding and Removing Columns

Adding a New Column

```
import pandas as pd

data = {"Name": ["Alice", "Bob", "Charlie"], "Marks": [85, 92, 78]}
df = pd.DataFrame(data)

# Add a new column with a list of values
df["Grade"] = ["A", "A+", "B+"]

# Add a column using calculation
df["Percentage"] = df["Marks"] / 100 * 100

print(df)
```

Figure 6.2 — Adding new columns to a DataFrame

Output

	Name	Marks	Grade	Percentage
0	Alice	85	A	85.0
1	Bob	92	A+	92.0
2	Charlie	78	B+	78.0

Removing Columns

```
# Remove a single column
df = df.drop("Percentage", axis=1)
```

```
# Remove multiple columns
df = df.drop(["Grade", "Marks"], axis=1)

print(df)
```

Figure 6.3 — Removing columns from a DataFrame



Note

The `axis=1` parameter tells pandas to operate on columns. `axis=0` (the default) operates on rows. This is a fundamental concept in pandas that you will encounter frequently.

6.3 Adding and Removing Rows

Adding Rows

```
import pandas as pd

data = {"Name": ["Alice", "Bob"], "Marks": [85, 92]}
df = pd.DataFrame(data)

# Add a new row using loc
df.loc[2] = ["Charlie", 78]

# Add using pd.concat
new_row = pd.DataFrame({"Name": ["David"], "Marks": [90]})
df = pd.concat([df, new_row], ignore_index=True)

print(df)
```

Figure 6.4 — Adding rows to a DataFrame

Output

	Name	Marks
0	Alice	85
1	Bob	92
2	Charlie	78
3	David	90

Removing Rows


```
# Remove row at index 1
df = df.drop(1)

# Remove multiple rows
df = df.drop([0, 2])

print(df)
```

Figure 6.5 — Removing rows from a DataFrame

6.4 Sorting a DataFrame

```
import pandas as pd

data = {"Name": ["Alice", "Bob", "Charlie", "David"], "Age": [21, 22, 20, 23], "GPA": [8.5, 7.9, 9.1, 8.0]}
df = pd.DataFrame(data)

# Sort by GPA in descending order
df_sorted = df.sort_values(by="GPA", ascending=False)
print(df_sorted)

# Sort by multiple columns
df_sorted2 = df.sort_values(by=["Age", "GPA"], ascending=[True, False])
print(df_sorted2)
```

Figure 6.6 — Sorting a DataFrame

Output

	Name	Age	GPA
2	Charlie	20	9.1
0	Alice	21	8.5
3	David	23	8.0
1	Bob	22	7.9

6.5 Handling Missing Data in a DataFrame

```
import pandas as pd
import numpy as np
```

```
data = {"Name": ["Alice","Bob","Charlie","David"], "Age": [21,np.nan,20,23],
"GPA": [8.5,7.9,np.nan,8.0]}
df = pd.DataFrame(data)

# Check for missing values
print(df.isnull())

# Count missing per column
print(df.isnull().sum())

# Fill missing values
df_filled = df.fillna({"Age": df["Age"].mean(), "GPA": 0})
print(df_filled)

# Drop rows with any missing values
df_clean = df.dropna()
print(df_clean)
```

Figure 6.7 — Handling missing data in a DataFrame

Method	Description
df.isnull()	Returns True/False for each cell (True = missing)
df.isnull().sum()	Counts missing values per column
df.fillna(value)	Replaces missing values with the specified value
df.dropna()	Removes rows containing any missing value
df.dropna(axis=1)	Removes columns containing any missing value

Chapter 7

Accessing DataFrames

Accessing specific rows, columns, or cells in a DataFrame is one of the most important skills in data handling. Pandas provides multiple methods to select exactly the data you need.

7.1 Selecting Columns

Selecting a Single Column

```
import pandas as pd

data = {"Name": ["Alice", "Bob", "Charlie", "David"], "Age": [21, 22, 20, 23],
        "Course": ["B.Tech CS", "B.Tech IT", "BCA", "B.Tech CS"], "GPA": [8.5, 7.9, 9.1, 8.0]}
df = pd.DataFrame(data)

# Select a single column (returns a Series)
print(df["Name"])

# Alternative dot notation
print(df.Name)
```

Figure 7.1 — Selecting a single column

Output

```
0    Alice
1     Bob
2  Charlie
3    David
Name: Name, dtype: object
```

Selecting Multiple Columns

```
# Select multiple columns (returns a DataFrame)
print(df[["Name", "GPA"]])
```

Figure 7.2 — Selecting multiple columns

Output

```
   Name  GPA
```

```

0    Alice  8.5
1      Bob  7.9
2  Charlie  9.1
3    David  8.0

```

7.2 Selecting Rows Using loc and iloc

loc

Label-based selection. Use loc when you want to access rows and columns by their labels (names). Syntax: `df.loc[row_label, column_label]`

iloc

Integer position-based selection. Use iloc when you want to access rows and columns by their numerical position (0, 1, 2...). Syntax: `df.iloc[row_position, column_position]`

Using loc (Label-Based)

```

import pandas as pd

data = {"Name": ["Alice", "Bob", "Charlie", "David"], "Age": [21, 22, 20, 23], "GPA": [8.5, 7.9, 9.1, 8.0]}
df = pd.DataFrame(data, index=["S001", "S002", "S003", "S004"])

# Select a single row by label
print(df.loc["S002"])

# Select multiple rows
print(df.loc[["S001", "S003"]])

# Select specific rows and columns
print(df.loc["S001":"S003", "Name":"Age"])

```

Figure 7.3 — Using loc for label-based selection

Output

```

Name    Bob
Age      22
GPA      7.9
Name: S002, dtype: object

      Name  Age  GPA

```

S001	Alice	21	8.5
S003	Charlie	20	9.1

	Name	Age
S001	Alice	21
S002	Bob	22
S003	Charlie	20

Using iloc (Position-Based)

```
# Select row at position 0
print(df.iloc[0])

# Select rows 0 to 2 and columns 0 to 1
print(df.iloc[0:3, 0:2])

# Select specific positions
print(df.iloc[[0, 3], [0, 2]])
```

Figure 7.4 — Using iloc for position-based selection

Output

```
Name    Alice
Age      21
GPA      8.5
Name: S001, dtype: object
```

	Name	Age
S001	Alice	21
S002	Bob	22
S003	Charlie	20

	Name	GPA
S001	Alice	8.5
S004	David	8.0

7.3 Conditional Selection (Filtering Rows)

One of the most powerful features of pandas is the ability to filter rows based on conditions.

```
import pandas as pd

data = {"Name": ["Alice", "Bob", "Charlie", "David", "Eve"], "Age": [21, 22, 20, 23, 21],
        "Course": ["B.Tech CS", "B.Tech IT", "BCA", "B.Tech CS", "BCA"], "GPA":
        [8.5, 7.9, 9.1, 8.0, 8.8]}
df = pd.DataFrame(data)

# Students with GPA above 8.0
print(df[df["GPA"] > 8.0])

# Students in B.Tech CS
print(df[df["Course"] == "B.Tech CS"])

# Multiple conditions (AND)
print(df[(df["GPA"] > 8.0) & (df["Age"] <= 21)])

# Multiple conditions (OR)
print(df[(df["Course"] == "BCA") | (df["GPA"] > 9.0)])
```

Figure 7.5 — Conditional filtering of a DataFrame

Operator	Meaning	Example
==	Equal to	df[df["Course"] == "BCA"]
!=	Not equal to	df[df["Course"] != "BCA"]
>	Greater than	df[df["GPA"] > 8.0]
<	Less than	df[df["Age"] < 22]
>=	Greater than or equal to	df[df["GPA"] >= 8.5]
<=	Less than or equal to	df[df["Age"] <= 21]
&	AND (both conditions true)	df[(cond1) & (cond2)]
	OR (either condition true)	df[(cond1) (cond2)]

⚠ Common Error

When using multiple conditions with & or |, always wrap each condition in parentheses. Writing `df[df["GPA"] > 8.0 & df["Age"] < 22]` without parentheses will cause an error.

7.4 Quick Reference — loc vs iloc

Feature	loc	iloc
Selection Type	Label-based	Position-based (integer)
Row Access	<code>df.loc["row_label"]</code>	<code>df.iloc[0]</code>
Column Access	<code>df.loc[:, "col_name"]</code>	<code>df.iloc[:, 0]</code>
Slicing	Inclusive of end label	Exclusive of end position
Use When	You know the label/name	You know the position/number

ETIHANS
CLOUD & AI ACADEMY



Chapter 8

Reading Data from CSV, Excel, and JSON Files

In real-world data analysis, you rarely type data manually. Instead, data is stored in files — CSV, Excel, JSON, and more. Pandas provides simple functions to load data from these files directly into a `DataFrame`.

8.1 Reading CSV Files

CSV	Comma-Separated Values. A plain text file where each line is a data record, and values are separated by commas. CSV is the most widely used format for tabular data.
------------	--

Suppose you have a file called **students.csv** with the following content:

```
Name, Age, Course, GPA
Alice, 21, B.Tech CS, 8.5
Bob, 22, B.Tech IT, 7.9
Charlie, 20, BCA, 9.1
David, 23, B.Tech CS, 8.0
Eve, 21, BCA, 8.8
```

Figure 8.1 — Sample CSV file content (*students.csv*)

Loading a CSV File

```
import pandas as pd

# Read the CSV file into a DataFrame
df = pd.read_csv("students.csv")

print(df)
```

Figure 8.2 — Reading a CSV file into a `DataFrame`

Output

	Name	Age	Course	GPA
0	Alice	21	B.Tech CS	8.5
1	Bob	22	B.Tech IT	7.9

2	Charlie	20	BCA	9.1
3	David	23	B.Tech CS	8.0
4	Eve	21	BCA	8.8

Useful Parameters for read_csv()

Parameter	Description	Example
sep	Specify a different separator	pd.read_csv("file.tsv", sep="\t")
header	Row number for column names (default 0)	pd.read_csv("file.csv", header=None)
index_col	Column to use as row index	pd.read_csv("file.csv", index_col="Name")
usecols	Read only specific columns	pd.read_csv("file.csv", usecols=["Name", "GPA"])
nrows	Read only first n rows	pd.read_csv("file.csv", nrows=100)
encoding	Specify file encoding	pd.read_csv("file.csv", encoding="utf-8")
na_values	Additional values to treat as NaN	pd.read_csv("file.csv", na_values=["NA", "-"])

Example: Reading CSV with Parameters

```
import pandas as pd

# Read CSV with Name as index, only specific columns
df = pd.read_csv("students.csv", index_col="Name", usecols=["Name", "Age",
"GPA"])

print(df)
```

Figure 8.3 — Using parameters with read_csv()

Output

	Age	GPA
Name		
Alice	21	8.5
Bob	22	7.9
Charlie	20	9.1
David	23	8.0
Eve	21	8.8

8.2 Reading Excel Files

Excel File	A spreadsheet file (.xlsx or .xls) created by Microsoft Excel. It can contain multiple sheets. Pandas can read data from any sheet.
------------	---

To read Excel files, pandas uses an additional library called **openpyxl**. Install it first:

```
pip install openpyxl
```

Figure 8.4 — Installing openpyxl for Excel support

Loading an Excel File

```
import pandas as pd

# Read an Excel file
df = pd.read_excel("students.xlsx")

print(df)
```

Figure 8.5 — Reading an Excel file into a DataFrame

Reading a Specific Sheet

```
# Read a specific sheet by name
df = pd.read_excel("students.xlsx", sheet_name="Sheet2")

# Read a specific sheet by position
df = pd.read_excel("students.xlsx", sheet_name=1)

# Read all sheets (returns a dictionary)
all_sheets = pd.read_excel("students.xlsx", sheet_name=None)
print(all_sheets.keys())
```

Figure 8.6 — Reading specific sheets from Excel

Useful Parameters for read_excel()

Parameter	Description	Example
-----------	-------------	---------

sheet_name	Name or index of the sheet	sheet_name="Sales Data"
header	Row to use as column headers	header=0
index_col	Column to use as row index	index_col="ID"
usecols	Columns to read (by name or letter)	usecols="A:D"
skiprows	Number of rows to skip from top	skiprows=2
nrows	Number of rows to read	nrows=50

8.3 Reading JSON Files

JSON	JavaScript Object Notation. A lightweight text format for storing data as key-value pairs. Commonly used in web applications and APIs.
-------------	--

Suppose you have a file called **students.json**:

```
[
  {"Name": "Alice", "Age": 21, "Course": "B.Tech CS", "GPA": 8.5},
  {"Name": "Bob", "Age": 22, "Course": "B.Tech IT", "GPA": 7.9},
  {"Name": "Charlie", "Age": 20, "Course": "BCA", "GPA": 9.1}
]
```

Figure 8.7 — Sample JSON file content (students.json)

Loading a JSON File

```
import pandas as pd

# Read a JSON file
df = pd.read_json("students.json")

print(df)
```

Figure 8.8 — Reading a JSON file into a DataFrame

Output

	Name	Age	Course	GPA
0	Alice	21	B.Tech CS	8.5
1	Bob	22	B.Tech IT	7.9
2	Charlie	20	BCA	9.1

JSON Orientations

Orientation	Structure	Example Usage
records	List of dictionaries (most common)	<code>pd.read_json("file.json", orient="records")</code>
columns	Dictionary of lists (column-oriented)	<code>pd.read_json("file.json", orient="columns")</code>
index	Dictionary of dictionaries (row-oriented)	<code>pd.read_json("file.json", orient="index")</code>
split	Separate lists for index, columns, data	<code>pd.read_json("file.json", orient="split")</code>

8.4 Comparison of File Formats

Feature	CSV	Excel	JSON
File Extension	.csv	.xlsx / .xls	.json
Human Readable	Yes (plain text)	No (binary)	Yes (plain text)
Multiple Sheets	No	Yes	No
Data Types Preserved	No (all text)	Yes	Partially
File Size	Small	Medium to Large	Medium
Best For	Simple tabular data	Complex formatted reports	Web APIs and nested data
Pandas Function	<code>pd.read_csv()</code>	<code>pd.read_excel()</code>	<code>pd.read_json()</code>



Pro Tip

For most data analysis tasks, CSV is preferred — simple, lightweight, and universally supported. Use Excel when you need multiple sheets or formatting. Use JSON when working with web APIs or nested data.

Chapter 9

Writing Data to Files

After loading, cleaning, and analyzing data, you often need to save the results. Pandas makes it easy to export your DataFrame to various file formats.

9.1 Writing to a CSV File

```
import pandas as pd

data = {"Name": ["Alice", "Bob", "Charlie", "David"], "Age": [21, 22, 20, 23], "GPA": [8.5, 7.9, 9.1, 8.0]}
df = pd.DataFrame(data)

# Write to CSV file
df.to_csv("output_students.csv", index=False)

print("File saved successfully!")
```

Figure 9.1 — Writing a DataFrame to a CSV file

The **index=False** parameter prevents pandas from writing the row index to the file. Without this, an extra column of numbers (0, 1, 2, 3) would be included.

Useful Parameters for to_csv()

Parameter	Description	Example
index	Include row index (default True)	index=False
sep	Separator character	sep="\t" (tab-separated)
columns	Specific columns to write	columns=["Name", "GPA"]
header	Include column names (default True)	header=False
encoding	File encoding	encoding="utf-8-sig"
mode	Write mode: w (overwrite) or a (append)	mode="a"

9.2 Writing to an Excel File

```
import pandas as pd

data = {"Name": ["Alice","Bob","Charlie","David"], "Age": [21,22,20,23], "GPA":
[8.5,7.9,9.1,8.0]}
df = pd.DataFrame(data)

# Write to Excel
df.to_excel("output_students.xlsx", index=False, sheet_name="Student Records")

print("Excel file saved!")
```

Figure 9.2 — Writing a DataFrame to an Excel file

Writing Multiple Sheets to One Excel File

You can write multiple DataFrames as separate sheets using **ExcelWriter**.

```
import pandas as pd

df_cs = pd.DataFrame({"Name": ["Alice","David"], "GPA": [8.5, 8.0]})
df_bca = pd.DataFrame({"Name": ["Charlie","Eve"], "GPA": [9.1, 8.8]})

# Write multiple sheets
with pd.ExcelWriter("course_report.xlsx") as writer:
    df_cs.to_excel(writer, sheet_name="B.Tech CS", index=False)
    df_bca.to_excel(writer, sheet_name="BCA", index=False)

print("Multi-sheet Excel file saved!")
```

Figure 9.3 — Writing multiple DataFrames to separate sheets

9.3 Writing to a JSON File

```
import pandas as pd

data = {"Name": ["Alice","Bob","Charlie"], "Age": [21,22,20], "GPA":
[8.5,7.9,9.1]}
df = pd.DataFrame(data)

# Write to JSON
df.to_json("output_students.json", orient="records", indent=4)
```

```
print("JSON file saved!")
```

Figure 9.4 — Writing a DataFrame to a JSON file

The **orient="records"** parameter saves data as a list of dictionaries. The **indent=4** parameter makes the file readable with proper formatting.

The output file will look like this:

```
[
  {
    "Name": "Alice",
    "Age": 21,
    "GPA": 8.5
  },
  {
    "Name": "Bob",
    "Age": 22,
    "GPA": 7.9
  },
  {
    "Name": "Charlie",
    "Age": 20,
    "GPA": 9.1
  }
]
```

Figure 9.5 — Content of the generated JSON file

9.4 Quick Reference — Writing Methods

Method	Format	Example
df.to_csv()	CSV	df.to_csv("output.csv", index=False)
df.to_excel()	Excel	df.to_excel("output.xlsx", index=False)
df.to_json()	JSON	df.to_json("output.json", orient="records")
df.to_html()	HTML Table	df.to_html("output.html")
df.to_clipboard()	System Clipboard	df.to_clipboard() (paste in Excel)

9.5 Complete Workflow Example

Let us put everything together with a complete workflow that reads data, processes it, and writes the results.

```
import pandas as pd

# Step 1: Read data from CSV
df = pd.read_csv("students.csv")
print("Original Data:")
print(df)
print()

# Step 2: Add a new column
df["Status"] = df["GPA"].apply(lambda x: "Distinction" if x >= 8.5 else "Pass")

# Step 3: Filter students with GPA above 8.0
top_students = df[df["GPA"] > 8.0]
print("Top Students:")
print(top_students)
print()

# Step 4: Sort by GPA descending
top_students = top_students.sort_values(by="GPA", ascending=False)

# Step 5: Save the results
top_students.to_csv("top_students.csv", index=False)
top_students.to_excel("top_students.xlsx", index=False)

print("Results saved to CSV and Excel!")
```

Figure 9.6 — Complete data handling workflow



Pro Tip

Always verify your output files after saving. Open the CSV in a text editor and the Excel file in a spreadsheet application to confirm the data is correct.

Chapter Summary

In this chapter, you have covered the essential foundations of data handling with the pandas library in Python. Here is a recap of the key concepts:

- **pandas** is a powerful Python library for working with structured (tabular) data, providing two core data structures: Series and DataFrame.
- **Installing pandas** is done using `pip install pandas`. It is imported using the standard convention: `import pandas as pd`.
- A **Series** is a one-dimensional labeled array — like a single column in a spreadsheet. It can be created from lists, dictionaries, or scalar values.
- **Handling Series** includes accessing elements, performing arithmetic and statistical operations, filtering data, and handling missing values.
- A **DataFrame** is a two-dimensional labeled data structure — like a complete spreadsheet. It is the primary object in pandas for data analysis.
- **Handling DataFrames** involves inspecting structure, adding/removing rows and columns, sorting, and managing missing data.
- **Accessing DataFrames** is done using column selection, `loc` (label-based), `iloc` (position-based), and conditional filtering.
- **Reading data** from files uses `pd.read_csv()`, `pd.read_excel()`, and `pd.read_json()`.
- **Writing data** to files uses `df.to_csv()`, `df.to_excel()`, and `df.to_json()`.